

---

# py-cidr Reference Documentation

*Release 5.0.2*

**Gene C**

**Aug 25, 2026**



## API REFERENCE GUIDE:

|          |                                      |           |
|----------|--------------------------------------|-----------|
| <b>1</b> | <b>py-cidr</b>                       | <b>1</b>  |
| 1.1      | Overview . . . . .                   | 1         |
| 1.2      | Key features . . . . .               | 1         |
| 1.3      | Manual . . . . .                     | 2         |
| 1.4      | Getting Started . . . . .            | 2         |
| 1.5      | More Detail . . . . .                | 2         |
| <b>2</b> | <b>Recent Changes</b>                | <b>5</b>  |
| <b>3</b> | <b>API Reference Guide</b>           | <b>7</b>  |
| 3.1      | PyCidr Class . . . . .               | 7         |
| 3.2      | Cidr Class . . . . .                 | 13        |
| 3.3      | File Utilities . . . . .             | 26        |
| 3.4      | Cidr Map . . . . .                   | 27        |
| <b>4</b> | <b>Migrating from Cidr to PyCidr</b> | <b>31</b> |
| <b>5</b> | <b>Appendix</b>                      | <b>33</b> |
| 5.1      | Installation . . . . .               | 33        |
| 5.2      | Dependencies . . . . .               | 33        |
| 5.3      | Philosophy . . . . .                 | 34        |
| 5.4      | License . . . . .                    | 34        |
| 5.5      | AI Tooling . . . . .                 | 34        |
|          | <b>Python Module Index</b>           | <b>35</b> |
|          | <b>Index</b>                         | <b>37</b> |



## 1.1 Overview

py-cidr is a python module with a variety of networking / CIDR tools.

## 1.2 Key features

- PyCidr (As of version 5.0) is now the preferred class.

Very fast version that uses compiled C-code to do most of the work.

Using compiled C-code is simple, clean, robust, very maintainable, is compiled with modern, restrictive compiler and linker flags.

All CIDR and IP addresses are strings.

Depends on two new packages:

- cidrtools-ffi: [github](#) and [AUR](#).
- cidrtools: [github](#) and [AUR](#).

*cidrtools-ffi* has python bindings to the *cidrtools* compiled library.

- Cidr class is still supported.

It uses Python's own *ipaddress* module.

We decided to leave the Cidr class untouched. The primary reason is that this makes it straightforward to compare with PyCidr.

So, at least for now, the performance boost requires migrating from Cidr to PyCidr. At some point, we may update the Cidr class to take advantage of the very fast *cidrtools* library as well.

- *py\_cidr* has four Classes : PyCidr, Cidr, CidrMap, CidrFile

- PyCidr does everything Cidr class does just a lot faster.

It is built on top of the compiled library *libcidrtools.so*. The API has changed a little from the *Cidr* class and is, hopefully, a little simpler and cleaner. One of the benefits (and drawbacks) of starting fresh.

- PyCidr and Cidr offer a number of useful operations on IP addresses and network blocks.

- \* Support for both IPv4 and IPv6
- \* Compact a list of CIDRs to smallest set of CIDR blocks
- \* Convert an range of IP addresses to a list of CIDRs
- \* Exclude one list of cidr blocks from another list.

- \* Split a CIDR block in a list of *smaller* CIDRs blocks with a larger prefix.
- \* Identify and validate
- \* more
- CidrFile reads and writes text files with lists of cidrs.
  - skips shell style comments when reading
  - Uses atomic file writes
- CidrMap is a way to map cidr strings to values.

For example mapping IP prefixes (cidr blocks) to their ISO country codes.

- File writes use locking to ensure multiple processes handle cached files correctly.
- Built on Patricia trees using the [patricia26](#) module. [patricia26](#) is extremely fast.

See API reference documentation for details.

## 1.3 Manual

There are pre-built versions of the manual which comes as a PDF file as well as an HTML version. The manual includes the API reference guide for all classes.

The docs are in `src/data/docs` and the Arch PKGUILD installs them to `/usr/share/py-cidr/docs`.

## 1.4 Getting Started

All git tags are signed with [arch@sapience.com](mailto:arch@sapience.com) key which is available via WKD or download from <https://www.sapience.com/tech>. Add the key to your package builder gpg keyring. The key is included in the Arch package and the `source=` line with `?signed` at the end can be used to verify the git tag. You can also manually verify the signature

## 1.5 More Detail

The following are part of the module.

### CidrMap Class

CidrMap provides an optimized tool to map a network-prefix to a value. The map may be saved to a (cache) file and reused.

To make use of the file cache, provide a `cache_directory` when instantiating the class.

Note that there are 2 typed of maps, non-compact and compact. The default and by far the most common is non-compact. Here, every (prefix, value) is added to the map.

If the data is available as a list of (prefix, value) tuples, then the best and fastest way is using very fast `bulk_insert` to get the data into the Patricia tree.

Bulk inserting is 2 - 3 times faster than inserting the data in a loop one at a time. For large maps it is hugely beneficial to use `bulk_intert()`.

For a compact map, effort is made to reduce redundant entries. For example if the map contains the (prefix, tuple) = ('10.0.0.0/22', 'net-1') and then a new tuple ('10.0.0.0/24', 'net-1') is added to the map, the new one is considered redundant since the network is a subnet of '10.0.0.0/22' and the value, 'net-1' is the same. If the value was different, then it is deemed not compactable. Note that `bulk_insert()` is not available with *compact* maps.

A *CidrMap* contains 2 separate maps one for each network family; one tree for IPv4 and separate one for IPv6.

This will create both an IPv4 and an IPv6 cache file in the given directory. The code is careful about reading and writing the cache files and uses locking and atomic writes to ensure the integrity of the data.

For example if application starts, reads cache, updates with new items and some time later saves the cache - the module will detect if the cache changed (by another process using same cache directory) since it was last read in, and merge its own changes with the changes in the cache file before writing out the updated cache. This should ensure nothing gets lost.

This was built originally for our firewall tool, where part of the data gathering component creates maps of network prefixes (blocks of IP addresses) to geolocated country codes and other useful information such as the ASN(s) the prefix belongs to.

When looking up a CIDR, there are 2 methods available. The first, *lookup\_lmp()* returns the (lmp\_prefix, val) pair where *lmp\_prefix* is the network with the longest matching cidr prefix. The longer the cidr prefix, the more specific the network. A /24 is more specific than a /22 for example.

The method *lookup\_all()* returns every matching prefix and its associated value. the LMP is always the first element in returned in the list.

Since parallelizing often provides decent speedups, *CidrMap* provides a mechanism to do that. It allows each separate process or thread to work with private (thread local) cache. Each of the private data caches can then be merged together by the top level process or thread.

This avoids multiple threads/processes writing to the same in memory data at the same time. This is done using the *CidrMap::merge()* method.

Additional details are available in the API reference documentation.

Methods provided:

- *CidrMap.add\_prefix\_val()*
- *CidrMap.add\_prefix\_vals()*
- *CidrMap.lookup\_lmp()*
- *CidrMap.lookup\_all()*
- *CidrMap.items()*
- *CidrMap.save\_cache()*
- *CidrMap.merge()*

Static functions: \* *CidrMap.create\_private\_cache()*

### PyCidr Class

See the API reference in the documentation for details. This class provides a suite of tools we found ourselves using often, so we encapsulated them in this class. All methods in the class are *@staticmethod* and thus no instance of the class is needed. Just call them as you would any function (*PyCidr.xxx()*).

IP addresses and networks are always represented as strings.

Here is a sample of some of the available functionality, see the API doc for complete documentation.

- *PyCidr.compact()*
- *PyCidr.cidr\_parts()*
- *PyCidr.exclude\_cidrs()*
- *PyCidr.is\_subnet()*
- *PyCidr.set\_prefix()*
- *PyCidr.sort()*

- `PyCidr.get_host_bits()`, `format_host_bits()`
- `PyCidr.cidr_to_range()`, `range_to_cidrs()`
- `PyCidr.fix_host_bits()`
- `PyCidr.clean_cidr()`, `clean_cidrs()`
- `PyCidr.split_by_ip_type()`
- `PyCidr.num_ips()`
- `PyCidr.ip_version()`
- `PyCidr.is_valid_ipv4()`, `is_valid_ipv6`, `is_valid_cidr()`
- `PyCidr.subnets_split()`

### CidrFile Class

This class provides a few reader/writer tools for files with lists of CIDR strings. Readers ignores comments. All methods are *@staticmethod* and thus no instance of the class is required. Simply use them as functions (*Cidr.xxx()*)

- `Cidr.read_cidr_file(file:str, verb:bool=False) -> [str]:`
- `Cidr.read_cidr_files(targ_dir:str, file_list:[str]) -> [str]`
- `Cidr.write_cidr_file(cidrs:[str], pathname:str) -> bool`
- `Cidr.read_cidrs(fname:str|None, verb:bool=False) -> (ipv4:[str], ipv6:[str]):`
- `Cidr.copy_cidr_file(src_file:str, dst_file:str) -> None`

### Cidr Class

Quite similar to `PyCidr` but not identical. See the API Reference Guide for more details.

## RECENT CHANGES

### 5.0.2

- New PyCidr class - replaces functionality in Cidr class. Using the pure C-code *cidrtools* library and its python bindings *cidrtools-cffi*
  - Superior performance. For example compacting lists of cidr blocks runs about 10 - 20 times faster.
  - API is similar but migrating the code will unlock the performance benefit.
  - Depends on two new packages: *cidrtools* and *cidrtools-cffi*
  - Updated build system using meson and uv.
- Cidr class still supported unchanged. This facilitates testing and comparing Cidr with PyCidr. In the future, some/all of Cidr may be migrated to use the new fast codebase.
- See manual for details and how to use the new speedy class!
- Add the manual



## API REFERENCE GUIDE

Cidr is the original class and over time it's API has grown in part to maintain backward compatibility and in part it has a dual personality. It is built on top of Python's native *ipaddress* class and ended up supporting network input/output in both string and *ipaddress* format.

PyCidr is designed for performance and simplicity. It only uses strings for IP address and CIDR blocks. The code is written in Cython/C-code and is significantly faster than the Cidr class variants. For example, our tests show compacting a list of CIDRs to the smallest number runs 18 to 22 times faster.

We recommend migrating to PyCidr. This is completely optional. The Cidr class remains supported. If you choose to migrate then it means using strings only (no *ipaddress*). If code is already strings the migration is pretty straight forward and there is a quick guide how to do so.

The next section has the auto-generated API reference.

### 3.1 PyCidr Class

#### 3.1.1 PyCidr: Overview

Before getting into detail, this is a brief summary of the member functions available. All member functions are *static* and therefore the member functions are called directly using `PyCidr.xxx()` - no need to instantiate the class.

- `compact()`

Takes a list of cidrs and returns a new list with the smallest number of cidrs that span exactly the space as the original list.

- `cidr_parts()`

Separates a cidr into the IP part and its prefix.

Example:

```
cidr_parts("10.0.0.0/24")    -> ("10.0.0.0", 24)
cidr_parts("10.0.0.27/32")   -> ("10.0.0.27", 32)
cidr_parts("2001:db8::1/64") -> ("2001:db8::", 64)
```

- `exclude_cidrs()`

Exclude one list of cidrs from another.

```
smaller = PyCidr.exclude_cidrs(all_cidrs, excluded_cidrs)
```

- `is_subnet()`

Determine if a cidr block is a subnet of any of a list of networks.

```
all_cidrs = [ '10.0.0.0/24', ... ]
is_subnet = PyCidr.is_subnet('10.0.0.22/32', all_cidrs)
```

- `set_prefix()`

Change the prefix (or add a prefix to an IP address).

```
cidr = '10.0.0.0/24'
new_cdr = PyCidr.set_prefix(cidr, 22)
```

- `sort()`

Sorts a list of network blocks.

```
sorted_items = PyCidr.sort(all_cidrs)
```

- `get_host_bits()`, `format_host_bits()`

Extracts the host bits of an IPv4 or IPv6 CIDR string and returns either an integer or human readable string from `format_host_bits`.

```
hbit_str = PyCidr.format_host_bits("10.0.0.22/24") # -> "0.0.0.22"
```

- `cidr_to_range()`, `range_to_cidrs()`

`cidr_to_range` returns the first, last and middle ip address of a cidr, while `range_to_cidrs()` generates the list of network cidrs that span a range of IPs from first to last.

- `fix_host_bits()`

Sometimes a cidr has host bits set, this o's them out. For example "10.0.0.22/24" would return "10.0.0.0/24"

- `clean_cidr()`, `clean_cidrs()`

Clean one cidr or a list of cidrs. A clean cidr is one that has 0 for host bits For example:

10.1.2.3/24 -> 10.1.2.0/24

- `split_by_ip_type()`

Takes a list of cidrs, can be mixed IPv4 and IPv6 and separates the IPv4 from the IPv6 blocks. It returns a tuple of [ipv4\_cidrs, ipv6\_cidrs]

- `num_ips()`

Returns the number of IPs in a cidr block.

- `ip_version()`

If cidr is IPV4 then this returns the integer 4. If an IPv6 then returns 6.

- `is_valid_ipv4()`, `is_valid_ipv6`, `is_valid_cidr()`

They each return True or False as appropriate.

- `subnets_split()`

Splits an IPv4 or IPv6 CIDR block into smaller subnets of a larger prefix size.

Examples:

```
subnets_split("10.0.0.0/23", 24) -> ["10.0.0.0/24", "10.0.1.0/24"]
```

PyCidr provides a suite of common network CIDR utilities

- PyCidr works exclusively with strings. This contrasts with the older Cidr class which also supports Python's `ipaddress`.

`class py_cidr.pycidr_class.PyCidr`

Bases: `object`

Provides a suite of network CIDR tools using the `cidrtools` compiled library.

All methods are static and are called without need to instantiate the class. For example:

```
new_cidrs = PyCidr.compact(cidrs)
```

It does not use or support any data types from `ipaddress` (far too slow)

`static cidr_parts(cidr: str) → tuple[str, int]`

Unified entry point splitting any valid IPv4 or IPv6 CIDR block into a clean base network address string and its integer prefix length.

### Examples

```
cidr_parts("10.0.0.22/24") -> ("10.0.0.0", 24) cidr_parts("2001:db8::1/64") -> ("2001:db8::", 64)
```

#### Parameters

`cidr` – The cidr string to examine

#### Returns

Tuple of (ip-address, prefix)

`static cidr_to_range(cidr: str) → tuple[str, str, str]`

Cidr string to an IP Range : (First, Middle, Last) IPs

#### Parameters

`cidr` – The cidr string to examine.

#### Returns

A tuple (ipo, mid, ip1) First, Middle and Last IP address of the cidr block.

`static cidrs_intersection(cidrs1: list[str], cidrs2: list[str]) → list[str]`

Returns the intersection subnets of 2 lists of cidrs.

#### Parameters

- `cidrs1` – First set of cidrs
- `cidrs2` – Second set of cidrs

#### Returns

List of the intersecting subnets (may be empty).

`static clean_cidr(cidr: str) → str`

Clean up a cidr address.

#### What it does:

- Ensures host bits are set to zero to match the prefix.
- Ensure the prefix is appropriate for the IP family, bounded by /32 or /128 for IPv4 or IPv6.
- Non-cidr (bad) strings are set to ''

#### Parameters

`cidr` – Cidr string to clean up.

**Returns**

cidr string if valid (empty if not valid)

```
static clean_cidrs(cidrs: list[str]) → list[str]
```

Takes a list of raw input CIDR strings, filters out any invalid items entirely, and returns a clean list of canonical networks with corrected host bits. Skips invalid entries completely instead of leaving them as empty strings.

**Parameters**

cidrs – List of cidr strings to be cleaned

**Returns**

New list of cidrs that are now clean. Bad cidrs are returned as empty strings.

```
static compact(cidrs: list[str]) → list[str]
```

Compact list of cidrs - can be mixed ipv4/ipv6. Returns 1 type, making type annotation simpler for caller.

**Parameters**

cidrs – Input list of cidr strings.

**Returns**

Compacted list of cidrs. If mixed ipv4 is before ipv6

```
static ct_version() → str
```

**Returns**

The cidrtools library version

```
static exclude_cidrs(all_cidrs: list[str], excluded_cidrs: list[str]) → list[str]
```

Return new list of cidrs made by removing all excluded\_cidrs from all\_cidrs. Safely segregates mixed stacks to prevent invalid cross-version failures.

**Parameters**

all\_cidrs – The full list of cidrs

**Excluded\_cidrs**

The cidrs to be removed from the full list

**Returns**

New list of cidrs after the excluded\_cidrs are removed.

```
static fix_host_bits(cidr: str) → str
```

Ensures that host bits in an IPv4 or IPv6 CIDR string are set to 0.

**Examples**

```
“10.0.0.22/32” -> “10.0.0.22/32” “10.0.0.22/24” -> “10.0.0.0/24”
```

A strictly valid cidr address must have host bits set to zero.

**Parameters**

cidr – The cidr to “fix” if needed.

**Returns**

The cidr with any non-zero host bits now zeroed out.

```
static format_host_bits(cidr: str) → str
```

Extracts the host bits of an IPv4 or IPv6 address by masking off the network bits defined by the prefix.

alias of get\_host\_bits().

**Parameters**

`cidr` – The cidr to examine.

**Returns**

String with network bits zero showing host bit.

```
static get_host_bits(cidr: str) → str
```

Extracts the host bits of an IPv4 or IPv6 address by masking off the network bits defined by the prefix.

**Parameters**

`cidr` – The cidr to examine.

**Returns**

String with network bits zero showing host bit.

```
static ip_type(cidr: str) → str
```

Determines if address string is valid ipv4 or ipv6 or not.

**Parameters**

`cidr` – ip address or cidr string

**Returns**

‘ip4’ or ‘ip6’ or empty string, ‘’, if invalid address

```
static ip_version(cidr: str) → int
```

Determines the IP version number (4 or 6 or 0 if neither)

**Parameters**

`cidr` – IP address or cidr string.

**Returns**

4 if IPv4, 6 if IPv6, 0 otherwise

```
static is_rfc_1918(cidr: str) → bool
```

Check if cidr is any RFC 1918.

**Parameters**

`cidr` – IP or Cidr to check if RFC 1918.

**Returns**

True if cidr is an RFC 1918 address, False if not.

```
static is_subnet(cidr: str, cidrs: list[str]) → bool
```

Check if cidr is a subnet of any of the list of IPvxNetworks .

**Parameters**

- `cidr` – Cidr string to check.
- `cidrs` – list of network blocks to check.

**Returns**

True if cidr is subnet of any of the cidrs list, else False.

```
static is_valid_cidr(cidr: str) → bool
```

Check if string is valid cidr or IP address (either IPv4 or IPv6) Equivalen to - `is_valid_ipv4(cidr)` or `is_valid_ipv6(cidr)`

**Param**

Address to check. Host bits set is permitted for a cidr network.

**Returns**

True/False if address is valid IPv4 or IPv6 address or network.

```
static is_valid_ipv4(cidr: str) → bool
```

Check if input is a valid IPv4 cidr or IP address

**Parameters**

cidr – Check if this cidr is valid.

**Returns**

True if valid IPv4 else False

```
static is_valid_ipv6(cidr: str) → bool
```

Check if input is a valid IPv6 cidr or IP address

**Parameters**

cidr – Check if this is a valid IPv6 address or cidr.

**Returns**

True if valid IPv6 else False

```
static num_ips(cidr: str) → int
```

Returns the number of IP addresses in the cidr block.

**Parameters**

cidr – The cidr to examine

**Returns**

The number of IP addresses in the range. For IPv6 This number is capped at  $2^{64}$  (per cidrtools library)

```
static range_to_cidrs(start_ip: str, end_ip: str) → list[str]
```

Generate a list of cidrs from an IP range.

**Parameters**

- start\_ip – First IP in range
- end\_ip – Last IP in range

**Returns**

list of cidr network blocks representing the IP range.

```
static remove_rfc_1918(cidrs: list[str]) → list[str]
```

Given list of cidrs, return list without any rfc 1918.

**Parameters**

cidrs – list of cidr strings.

**Returns**

List of cidrs with any RFC 1918 addresses removed.

```
static rfc_1918_cidrs() → list[str]
```

Return list of rfc 1918 networks cidr strings

**Returns**

list of RFC 1918 networks as cidr strings

```
static set_prefix(cidr: str, prefix: int) → str
```

Set new prefix for cidr and return new cidr string.

**Parameters**

cidr – Cidr string to use

**Prefix**

The new prefix to use

**Returns**

Cidr string using the specified prefix

```
static sort(cidrs: list[str]) → list[str]
```

Sort the list of cidr strings.

**Parameters**

cidrs – list of cidrs.

**Returns**

Sorted copy of cidr list

```
static split_by_family(cidrs: list[str]) → tuple[list[str], list[str]]
```

Split a list of cidrs into ipv4 and ipv6 lists.

**Parameters**

cidrs – list of cidr strings

**Returns**

Tuple of an ipv4 cidr list and an ipv6 cidr list

```
static split_by_iptype(cidrs: list[str]) → tuple[list[str], list[str]]
```

Split a list of cidrs into ipv4 and ipv6 lists. Historical Alias of split\_by\_family()

**Parameters**

cidrs – list of cidr strings

**Returns**

Tuple of an ipv4 cidr list and an ipv6 cidr list

```
static subnets_split(cidr: str, new_prefix: int) → list[str]
```

Splits an IPv4 or IPv6 CIDR block into smaller subnets of a larger prefix size. If the new prefix is smaller or equal to the existing one, returns the original CIDR.

**Examples**

```
subnets_split("10.0.0.0/23", 24) -> ["10.0.0.0/24", "10.0.1.0/24"]
```

**Parameters**

- cidr – the cidr to work on
- new\_prefix – the prefix to use to split cidr into smaller subnets

**Returns**

list of subnets - if compacted will be the input cidr block.

```
static version() → str
```

**Returns**

Returns the py-cidr Version

## 3.2 Cidr Class

Class providing some common CIDR utilities

```
class py_cidr.cidr_class.Cidr
```

Bases: object

Provides suite of CIDR tools.

All methods are (static) and are thus called without need to instantiate the class. For example:

```
net = Cidr.cidr_to_net(cidr_string)
```

**Notation:**

- cidr means a string
- net means ipaddress network (IPv4Network or IPv6Network)
- ip means an IP address string
- addr means an ip address (IPv4Address or IPv6Address)
- address means either a IP address or a cidr network as a string

```
static address_iptype(addr: IPvxAddress | IPvxNetwork) → str
```

Identify address or net (IPvxNetwork) as ipv4, ipv6 or neither.

**Parameters**

- addr (*str*)
- . (*ipaddress IP or network*)

**Returns**

'ip4', 'ip6' or ''

**Return type**

str | None

```
static address_to_net(addr: IPAddress | IPvxNetwork, strict: bool = False) → IPvxNetwork | None
```

Convert an address to IPvxNetwork.

Be flexible with input address. Can be IPAddress (includes string) or even IPvxNetwork.

**Parameters**

- addr (*IPAddress | IPvxNetwork*) – Input address.
- strict (*bool*) – If true then cidr is considered invalid if host bits are set. Defaults to False. (see ipaddress docs).

**Returns**

The IPvxNetwork derived from input "addr" or None if not an address/network.

**Return type**

IPvxNetwork | None

```
static addresses_to_ips(addresses: list[IPvxAddress]) → list[str]
```

From list of IPs in ipaddress format, get list of ip strings.

**Parameters**

- addresses (*list[IPvxAddress]*)
- format (*list of IP addresses in ipaddress*)

**Returns**

list of IP strings

**Return type**

list[str]

```
static cidr_exclude(cidr1: str, cidrs2: list[str]) → list[str]
```

Exclude cidr1 from any of networks in cidrs2.

**Parameters**

- `cidr1` (*str*)
- `excluded.` (*list of cidrs from which cidr1 will be*)
- `cidrs2` (*list[str]*)
- `excluded.`

**Returns**

Resulting list of cidrs (“cidrs2” - “cidr1”)

**Return type**

`list[str]`

```
static cidr_ipctype(address: Any) → str
```

Determines if address string is valid ipv4 or ipv6 or not.

**Parameters**

`address` – address or cidr string

```
static cidr_is_subnet(cidr: str, ipa_nets: list[IPvxNetwork]) → bool
```

Check if cidr is a subnet of any of the list of IPvxNetworks .

**Parameters**

- `cidr` (*str*)
- `check.` (*list of IPvxNetworks to*)
- `ipa_nets` (*list[IPvxNetwork]*)
- `check.`

**Returns**

True if cidr is subnet of any of the ipa\_nets, else False.

**Return type**

`bool`

```
static cidr_list_compact(cidrs: list[str], string: bool = True) → list[str] | list[IPvxNetwork]
```

Compact list of cidr networks to smallest list possible. Deprecated - use `compact_cidrs(cidrs, return_nets)` instead, it is the same with the boolean flag reversed.

**Parameters**

- `cidrs` (*list[str]*)
- `compact.` (*list of cidr strings to*)
- `string` (*bool*) –
  - If True (default), then return is a list of strings.
  - If False, a list of IPvxNetworks.

**Returns**

Compressed list of cidrs as ipaddress networks (`string=False`) or list of strings when `string=True`

**Return type**

`list[str] | list[IPvxNetwork]`

```
static cidr_range_split(cidr: str) → tuple[str, str, str]
```

Convert network to IP Range.

**Parameters**

- net (*IPvxNetwork*)
- network (*The*)
- string (*bool*)
- IPvxAddress (*If True then returns cidr strings instead of*)

**Returns**

that are strings of the first IP, middle IP and last IP in the cidr block

**Return type**

tuple[str, str, str]

```
static cidr_set_prefix(cidr: str, prefix: int) → str
```

Set new prefix for cidr and return new cidr string.

**Parameters**

- cidr (*str*)
- use (*The new prefix to*)
- prefix (*int*)
- use

**Returns**

Cidr string using the specified prefix

**Return type**

str

```
static cidr_to_net(cidr: str, strict: bool = False) → IPvxNetwork | None
```

Convert cidr string to ipaddress network.

**Parameters**

- cidr (*str*)
- string (*Input cidr*)
- strict (*bool*)
- set. (*If true then cidr is considered invalid if host bits are*)
- False. (*Defaults to*)

**Returns**

The ipaddress network derived from cidr string as IPvxNetwork = IPv4Network or IPv6Network or None if invalid.

**Return type**

IPvxNetwork | None

```
static cidr_to_range(cidr: str, string: bool = False) → tuple[IPvxAddress | str | None, IPvxAddress | str | None]
```

Cidr string to an IP Range.

**Parameters**

- `cidr(str)`
- `examine.` (*The cidr string to*)
- `string(bool)`
- `IPvxAddress` (*If True then returns cidr strings instead of*)

**Returns**

tuple (ip0, ip1) of first and last IP address in net (ip0, ip1) are IPvxAddress or str when string is True

**Return type**

tuple[IPAddress, IPAddress]

`static cidr_type_network(cidr: str) → tuple[str, type[IPvxNetwork]]`

Cidr Network Type.

**Parameters**

- `cidr(str)`
- `examine(Cidr string to)`

**Returns**

tuple(ip-type, net-type). ip-type is a string ('ip4', 'ip6') while network type is IPv4Network or IPv6Network

**Return type**

tuple[str, IPvxNetwork]

`static cidrs2_minus_cidrs1(cidrs1: list[str], cidrs2: list[str]) → list[str]`

Exclude all of cidrs1 from cidrs2.

i.e. return "cidrs2" - "cidrs1".

**Parameters**

- `cidrs1(list[str])`
- `excluded.` (*list of cidr strings from which cidrs1 are*)
- `cidrs2(list[str])`
- `excluded.`

**Returns**

Resulting list of cidr strings = "cidrs2" - "cidrs1".

**Return type**

list[str]

`static cidrs_exclude(cidrs1: list[str], cidrs2: list[str]) → list[str]`

Deprecated: replaced by `cidrs2_minus_cidrs1()`

`static cidrs_split_type(cidrs: list[str]) → tuple[list[str], list[str], list[str]]`

Split a list of cidrs into ipv4, ipv6 and other.

**Parameters**

`cidrs(list[str])` - list of cidr strings

**Returns**

list[str], ip6: list[str], other: list[str]] Tuple of lists of ipv4, ipv6 and unknown.

**Return type**

tuple[ip4

```
static cidrs_to_nets(cidrs: list[str], strict: bool = False) → list[IPvxNetwork]
```

Convert list of cidr strings to list of IPvxNetwork.

**Parameters**

- `cidrs` (*list[str]*)
- `strings` (*list of cidr*)
- `strict` (*bool*)
- `true` (*If*)
- `false`. (*cidr with host bits set is invalid. Defaults to*)

**Returns**

list of IPvxNetworks generated from cidrs.

**Return type**

list[IPvxNetwork]

```
static clean_cidr(cidr: str) → str | None
```

Clean up a cidr address.

**Does:**

- fix up host bits to match the prefix
- convert old class A,B,C style IPv4 addresses to cidr.

e.g.

a.b.c -> a.b.c.0/24 a.b.c.23/24 -> a.b.c.0/24

**Parameters**

- `cidr` (*str*)
- `up`. (*Cidr string to clean*)

**Returns**

- cidr string if valid
- None if cidr is invalid.

**Return type**

str | None

```
static clean_cidrs(cidrs: list[str]) → list[str]
```

Clean list of cidrs.

Similar to clean\_cidr() but for a list.

**Parameters**

- `cidrs` (*list[str]*)
- `up`. (*list of cidr strings to clean*)

**Returns**

list of cleaned cidrs. If input cidr is invalid then its returned as None

**Return type**

list[str]

```
static compact(cidrs: list[str]) → list[str]
```

Compact list of cidrs - can be mixed ipv4/ipv6. Returns 1 type, making type annotation simpler for caller.

**Parameters**

*cidrs (list[str])* – Input list of cidr strings.

**Returns**

Compacted list of cidrs. If mixed ipv4 is before ipv6

**Return type**

list[str]

Recommended methods are compact() or compact\_nets(). Others are kept for backward compatibility.

```
static compact_cidrs(cidrs: list[str], nets: bool = False) → list[str] | list[IPvxNetwork]
```

Compact a list of cidr networks as strings.

**Parameters**

- *cidrs (list[str])*
- *compact. (list of cidrs to)*
- *nets (bool)*
- *False (If)*
- *default (the)*
- *strings (the result will be list of)*
- *IPvxNetwork's. (else a list of)*

**Returns**

A list of compacted networks whose elements are strings if return\_nets is False or IPvxNetworks if True.

**Return type**

list[str | IPvxNetwork]

```
static compact_cidrs_to_cidrs(cidrs: list[str]) → list[str]
```

Compact list of cidr networks and return list of cidr strings

Same as compact\_cidrs(cidrs, nets=False). With single return type this is helpful when using type annotation checkers.

```
static compact_nets(nets: list[IPvxNetwork]) → list[IPvxNetwork]
```

Compact list of IPvxNetwork.

**Parameters**

- *nets (list[IPvxNetwork])*
- *compact. (Input list if networks to)*

**Returns**

Compacted list of IPvxNetworks.

**Return type**

list[IPvxNetwork]

```
static fix_cidr_host_bits(cidr: str, verb: bool = False) → str
```

zero out any host bits.

A strictly valid cidr address must have host bits set to zero.

**Parameters**

- `cidr(str)`
- `needed.` (*The cidr to "fix" if*)
- `verb(bool)`
- `False.` (*Some info on stdout when set True. Defaults to*)

**Returns**

The cidr with any non-zero host bits now zeroed out.

**Return type**

str

```
static fix_cidrs_host_bits(cidrs: list[str], verb: bool = False) → list[str]
```

zero any host bits for a list of cidrs.

Similar to `fix_cidr_host_bits()` but for a list of cidrs.

**Parameters**

- `cidrs(list[str])`
- `up.` (*list of cidrs to fix*)
- `verb(bool)`
- `False.` (*Some info on stdout when set True. Defaults to*)

**Returns**

The list of cidrs each with any non-zero host bits now zeroed out.

**Return type**

list[str]

```
static get_host_bits(ip: str, pfx: int = 24) → int
```

Gets the host bits from an IP address given the netmask.

**Parameters**

- `ip(str)`
- `examine.` (*The IP to*)
- `pfx(int)`
- `prefix.` (*The cidr*)

**Returns**

The host bits from the IP.

**Return type**

int

```
static ip_to_address(ip: str) → IPvxAddress | None
```

Return ipaddress of given ip.

If IP has prefix or host bits set, strip the prefix and keep host bits.

**Parameters**

- `ip(str)`
- `convert` (*The IP string to*)
- `Rreturns` (*IPvxAddress | None*)

- `address`. (*IPvxAddress derived from IP or None if not an IP*)

`static ip_version(addr_or_net: str | IPvxNetwork | IPvxAddress) → int`

Determines the IP version of an address or network:

**Parameters**

`addr_or_net` (*str | IPvxNetwork | IPvxAddress*) – Can be IP address or network either a string or ip\_address format.

**Returns**

4 if `addr_or_net` is IPv4Network or IPv4Address 6 if `addr_or_net` is IPv6Network or IPv6Address 0 otherwise

**Return type**

int

`static ipaddr_cidr_from_string(address: str, strict: bool = False) → IPvxNetwork | None`

Convert string of IP address or cidr net to IPvxNetwork

**Parameters**

- `address`
- `string`. (*IP or CIDR network as a*)
- `strict` (*bool*)
- `true` (*If*)
- `block`. (*host bits are disallowed for cidr*)

**Returns**

An IPvxNetwork or None if invalid.

**Return type**

IPvxNetwork | None

`static ips_to_addresses(ips: list[str]) → list[IPvxAddress]`

Convert list of IP strings to a list of ip addresses

**Parameters**

- `ips` (*list[str]*)
- `convert` (*list of IP strings to*)

**Returns**

list of IPvxAddress derived from input IPs.

**Return type**

list[IPvxAddress]

`static is_rfc_1918(cidr: str) → bool`

Check if cidr is any RFC 1918.

**Parameters**

- `cidr` (*str*)
- `1918`. (*IP or Cidr to check if RFC*)

**Returns**

True if cidr is an RFC 1918 address. False if not.

**Return type**

bool

```
static is_valid_cidr(address: Any) → bool
```

Check if address is a valid ip or cidr network.

**Parameters**

- address (*Any*)
- network. (*Address to check. Host bits set is permitted for a cidr*)

**Returns**

True/False if address is valid IPv4 or IPv6 address or network.

**Return type**

bool

```
static is_valid_ip4(address: Any) → bool
```

check if valid IPv4 address or cidr.

**Parameters**

- address (*Any*)
- cidr. (*Check if this is a valid IPv4 address or*)

**Returns**

True if valid IPv4 else False

**Return type**

bool

```
static is_valid_ip6(address: Any) → bool
```

check if valid IPv6 address or cidr.

**Parameters**

- address (*Any*)
- cidr. (*Check if this is a valid IPv6 address or*)

**Returns**

True if valid IPv6 else False

**Return type**

bool

```
static net_exclude(net1: IPvxNetwork, nets2: list[IPvxNetwork]) → list[IPvxNetwork]
```

Exclude net1 from any of networks in net2 and return resulting list of nets (without net1).

**Parameters**

- net1 (*IPvxNetwork*)
- ecluded. (*Network to be*)
- nets2 (*list[IPvxNetwork]*)
- excluded (*list of networks from which net1 will be*)
- from.

**Returns**

Resultant list of networks “nets2 - net1”.

**Return type**

list[IPvxNetwork]

```
static net_is_subnet(net1: IPvxNetwork, net2: IPvxNetwork | list[IPvxNetwork]) → bool
```

Determines if *net1* is a subnet of any of *net2*.

**Parameters**

- *net1* (*IPvxNetwork*) – Network to check if is a subnet.
- *net2* (*IPvxNetwork* | *list[IPvxNetwork]*) – Network or list of networks to be checked.

**Returns**

True if *net1* is a subnet of any of *net2*.

**Return type**

bool

```
static net_range_split(net: IPvxNetwork) → tuple[IPvxAddress, IPvxAddress, IPvxAddress]
```

Convert network to IP Range.

**Parameters**

- *net* (*IPvxNetwork*)
- *network* (*The*)
- *string* (*bool*)
- *IPvxAddress* (*If True then returns cidr strings instead of*)

**Returns**

that are the first IP, middle IP and last IP in the cidr block

**Return type**

tuple[, IPvxAddressIPAddress, , IPvxAddressIPAddress, IPvxAddress]

```
static net_to_cidr(net: IPvxNetwork) → str
```

**Net to Cidr String**

Convert an ipaddress network to a cidr string.

**Parameters**

*net* (*IPvxNetwork*) – Ipaddress Network to convert.

**Returns**

Cidr string from *net*. If unable to conver, then empty string is returned.

**Return type**

str

```
static net_to_range(net: IPvxNetwork, string: bool = False) → tuple[IPvxAddress | str | None,  
                                                                    IPvxAddress | str | None]
```

Convert network to IP Range.

**Parameters**

- *net* (*IPvxNetwork*)
- *network* (*The*)
- *string* (*bool*)
- *IPvxAddress* (*If True then returns cidr strings instead of*)

**Returns**

tuple (ip0, ip1) of first and last IP address in net Each (ip0, ip1) is IPvxAddress or a string if “string” == True

**Return type**

tuple[IPAddress, IPAddress]

`static nets_exclude(nets1: list[IPvxNetwork], nets2: list[IPvxNetwork]) → list[IPvxNetwork]`

Exclude every nets1 network from from any networks in nets2.

Similar to net\_exclude() except this version has a list to be excluded instead of a single network.

**Parameters**

- `nets1` (*list[IPvxNetwork]*)
- `excluded.` (*list of nets to be*)
- `nets2` - (*list[IPvxNetwork]*):
- `nets1.` (*list of nets from which will exclude any of*)

**Returns**

list of resultant networks (“nets2” - “nets1”)

**Return type**

list[IPvxNetwork]

`static nets_to_cidrs(nets: list[IPvxNetwork]) → list[str]`

Convert list of ipaddress networks to list of cidr strings.

**Parameters**

- `nets` (*list[IPvxNetwork]*)
- `convert.` (*list of nets to*)

**Returns**

list of cidr strings.

**Return type**

list[str]

`static range_to_cidrs(addr_start: IPAddress, addr_end: IPAddress, string: bool = False) → list[IPvxNetwork] | list[str]`

Generate a list of cidr/nets from an IP range.

**Parameters**

- `addr_start` (*IPAddress*)
- `range` (*End of IP*)
- `addr_end` (*IPAddress*)
- `range`
- `string` (*bool*)
- `IPvxNetwork` (*If True then returns list of cidr strings otherwise*)

**Returns**

list[IPvxNetwork] | list[str] list of cidr network blocks representing the IP range. list elements are IPvxAddress or str if parameter string=True

```
static remove_rfc_1918(cidrs_in: str | list[str]) → tuple[str | list[str], str | list[str]]
```

Given list of cidrs, return list without any rfc 1918

**Parameters**

- `list[str] (cidrs_in (str |))`
- `strings. (Cidr string or list of cidr)`

**Returns**

Returns (tuple[cidrs\_cleaned, rfc\_1918\_cidrs\_found]):

- `cidrs_cleaned`: list of cidrs with all rfc\_1918 removed.
- `rfc_1918_cidrs_found`: list of any rfc 1918 found in the input.

If input cidr(s) is a list, then items in output are a (possibly empty) list If not a list then returned items will be string or None.

**Return type**

tuple[str | list[str], str | list[str]]

```
static rfc_1918_cidrs() → list[str]
```

Return list of rfc 1918 networks cidr strings

**Returns (list[str]):**

list of RFC 1918 networks as cidr strings

```
static rfc_1918_nets() → list[IPvxNetwork]
```

Return list of rfc 1918 networks

**Returns**

list of RFC 1918 networks.

**Return type**

list[IPv4Network]

```
static sort_cidrs(cidrs: list[str]) → list[str]
```

Sort the list of cidr strings.

**Parameters**

- `cidrs (list[str])`
- `cidrs. (list of)`

**Returns**

Sorted copy of cidr list

**Return type**

list[str]

```
static sort_ips(ips: list[str]) → list[str]
```

Sort a list of IP addresses.

**Parameters**

- `ips (list[str])`
- `sorted. (list of ips to be)`

**Returns**

Sorted copy of ips.

**Return type**

list[str]

`static sort_nets(nets: list[IPvxNetwork]) → list[IPvxNetwork]`

Sort a list of networks.

**Parameters**`nets (list[IPvxNetwork])` – list of networks to be sorted.**Returns**

Sorted copy of networks.

**Return type**

list[IPvxNetwork]

`static version() → str`**Returns**

Version of py-cidr

### 3.3 File Utilities

CidrFile class: Read/write a file with list of cidr blocks as strings For reading

- comments ignored
- pname = is path to the file.
- cidr are all in column 1

`class py_cidr.cidr_file_class.CidrFile`

Bases: object

Some CIDR string file read/writ tools.

All methods are static so no class instance variable needed.

`static copy_cidr_file(src_file: str, dst_file: str) → bool`

Copy one file to another.

**Parameters**

- `src_file` – Source file to copy.
- `dst_file` – Destination Where to save a copy

**Returns**

True if all okay else False

`static read_cidr_file(fname: str | None, verb: bool = False) → list[str]`

Read file of cidrs and return list of all IPv4 and IPv6.

See `read_cidrs()` to have the cidrs split by IP family.**Parameters**

- `fname` – Path to file of cidrs to read. Stdin if set to None
- `verb` – More verbose output

**Returns**

list of all cidrs (ip4 and ip6) read from the file

```
static read_cidr_files(targ_dir: str, file_list: list[str]) → list[str]
```

Read files in a directory and return merged list of cidr strings.

**Parameters**

- *targ\_dir* – Directory to find each file.
- *file\_list* – read from this list of files found in in *targ\_dir*

**Returns**

list of all cidrs found in the files.

```
static read_cidrs(fname: str | None, verb: bool = False) → tuple[list[str], list[str]]
```

Read file of cidrs and return tuple of separate lists (ip4, ip6).

- if *fname* is *None* or *sys.stdin* then data is read from *stdin*.
- only column 1 of file is used.
- comments are ignored

See also *read\_cidr\_file()* which returns all cidrs in one list.

**Parameters**

- *fname* – File name to read. If *None*, then reads from *stdin*.
- *verb* – More verbose output when set to *True*.

**Returns**

A tuple of lists (*ipv4*, *ipv6*) of cidrs separated into IPv4 and IPv6.

```
static write_cidr_file(cidrs: list[str], pname: str) → bool
```

Write list of cidrs to a file.

**Parameters**

- *cidrs* – list of cidr strings to write.
- *pname* – Path to file where cidrs are to be written.

**Returns**

True if successful otherwise False.

## 3.4 Cidr Map

This tool uses Patricia Tree to maintain a map of cidr -> <value>. For example a map of all network prefixes where the <value> might be the country that network is located or registered in. Map cidr prefixes to some value.

Uses separate maps for ipv4 and ipv6

```
class py_cidr.cidr_map.CidrMap(cache_dir: str = "", compact: bool = False)
```

Bases: object

Class provides *map(cidr) -> some value*.

- *ipv4* and *ipv6* are cached separately
- built on *CidrCache* and *Cidr* classes

**Parameters**

- *cache\_dir* – Optional directory to keep cached copy of map

- `compact` – Optional flag to request a compact map. The standard map keeps every prefix added to map and this is the default. If compacting, then adjacent prefixes whose associated values are the same, will have the prefixed compacted to a single prefix.

`add_cidr(cidr: str, val: Any, priv_maps: PrefixMaps | None = None)`

Deprecated Legacy function - use `add_prefix_val()` instead.

`add_cidrs(prefixes: list[str], vals: list[Any])`

Deprecated Legacy function - use `add_prefix_vals()` instead.

`add_prefix_val(prefix_val: PrefixVal, priv_maps: PrefixMaps | None = None)`

Add cidr to the map..

#### Parameters

- `prefix_val` – The tuple of (prefix, value) to add to the map.
- `priv_map` – Optional private cache. If using multiple processes/threads then provide this object where changes are kept thread local to the private map instead of in the instance cache. This way the same instance (and its cache) can be used across multiple processes/threads. All the thread local caches can be merged ex post.

Use `CidrMap.create_private_cache()` to create private\_data

`add_prefix_vals(prefix_vals: list[PrefixVal])`

Add list if (prefix, val) tuples.

#### Parameters

`prefix_vals` – Add a list of tuples of (prefix, value) to map.

`static create_private_cache() → PrefixMaps`

Create and Return private cache object to use with `add_cidr()`.

This cache has no `cache_dir` set - memory only. Required if one `CidrMap` instance is used in multiple processes/threads Give each process/thread a private data cache and they can be merged into the `CidrMap` instance after they have all completed.

#### Returns

A `private_cache_data` object.

`items(v6: bool = False) → Iterator[tuple[str, Any]]`

Iterator that returns oen tuple element (prefix, value) at a time fomr the map.

#### Prefix v6

Default is false, and the elements are from IPv4 map If True, then elements are taken from the IPv6 map.

#### Returns

One tuple (prefix, value):

`lookup(cidr: str) → Any | None`

Return the value associated with the LPM (longest prefix match) of cidr. :param cidr: The cidr to lookup  
:returns: The value of the LPM of input cidr.

`lookup_all(cidr: str) → list[tuple[str, Any]]`

If cidr is in the map, return list of all (prefix, val) tuples. cidr is a subnet of each prefix. The first (prefix, val) returned is always the longest matching prefix (LMP) and it's value: (lmp, val)

The remaining elements will all have shorter prefix length (larger, less specific) network blocks.

**Parameters**

`cidr` – Cidr value to lookup.

**Returns**

A list of tuples (prefix, value)]. where cidr is same as or subnet of each prefix. Empty list If no matching elements found

`lookup_both(cidr: str) → tuple[str, Any]`

Deprecated: Historical - same as `lookup_lmp()`

`lookup_lmp(cidr: str) → tuple[str, Any]`

Check if cidr is in the map. Similar to `lookup()` but returns the longest matching prefix (LMP) and it's value. cidr is then the same as or subnet of prefix. If not found then empty string for prefix.

See `lookup_all()` which returns list of (prefix, val) tuples where the first element is the (lmp, val) pair.

**Parameters**

`cidr` – Cidr value to lookup.

**Returns**

A tuple of (prefix, value) where cidr is same as or a subnet of prefix and value is the associated value. If not found then prefix is empty string.

`merge(priv_maps: PrefixMaps | None)`

Merge private maps back into our instance map.

**Parameters**

`priv_maps` – The “private data” to add to the map. Merge the content of `priv_maps` into the current data. See `CidrMap.create_private_cache()`

`print()`

Print the cache data.

`save_cache()`

Write cache to files



## MIGRATING FROM CIDR TO PYCIDR

If existing code uses string representations, it is straightforward. If using `ipaddress` format, then code will need to be migrated to strings to get the performance benefits. `Cidr` class is not going away, so migrating is optional.

Note that for any functions with an argument to choose the output in string or `ipaddress` format the `PyCidr` replacement has no such argument because strings are always the output form.

Some examples migrating:

- Compact list of cidrs.

```
Cidr.compact(cids)
```

```
PyCidr.compact(cidrs)
```

`Cidr` has some additional forms all of which are replaced as above

- `Cidr` to IP address range

```
(ip_start, ip_end) = Cidr.cidr_to_range(cidr)
```

```
(ip_start, ip_mid, ip_end) = Cidr.cidr_range_split(cidr)
```

```
(ip_start, ip_mid, ip_end) = PyCidr.cidr_to_range(cidr)
```

- Sorting

While `Cidr` distinguished IP addresses from CIDR blocks and therefore provided separate functions for some tasks like sorting, `PyCidr` accepts an IP address (e.g. `10.0.0.22`) or a CIDR (e.g. `10.0.0.0/24` or `10.0.0.22/32`) everywhere.

```
x = Cidr.sort_cidrs(cidrs)
```

```
x = Cidr.sort_ips(ips)
```

```
x = PyCidr.sort(cidrs)
```

```
x = PyCidr.sort(ips)
```

However please note that `PyCidr` returns IP addresses with their prefix (`/32` for IPv4 and `/128` for IPv6). To recover just the IP part using `cidr_parts()`

```
(ip_part, pfx_part) = PyCidr.cidr_parts(cidr)
```

- Checking ip/cidr validity

```
Cidr.is_valid_ip4()  
Cidr.is_valid_ip6()  
Cidr.is_valid_cidr()  
  
PyCidr.is_valid_ipv4()  
PyCidr.is_valid_ipv6()  
PyCidr.is_valid_cidr()
```

## 5.1 Installation

Available on \* [Github](#) \* [Archlinux AUR](#)

On Arch you can build using the provided PKGBUILD in the packaging directory or from the AUR. To build manually, clone the repo and :

```
rm -f dist/*
./scripts/do-build
./scripts/do-install [destination]
```

The default destination is *build/pkg*.

## 5.2 Dependencies

**Run Time :**

- python (3.14 or later)
- lockmgr
- patricia26

**Building Package :**

- cython
- gcc
- git
- meson
- meson-python
- uv
- python-uv-build
- rsync
- python-pytest
- python-pytest-asyncio

## 5.3 Philosophy

We follow the *live at head commit* philosophy as recommended by Google's Abseil team<sup>1</sup>. This means we recommend using the latest commit on git master branch.

## 5.4 License

Created by Gene C. and licensed under the terms of the GPL-2.0-or-later license.

- SPDX-License-Identifier: GPL-2.0-or-later
- SPDX-FileCopyrightText: © 2024-present Gene C <[arch@sapience.com](mailto:arch@sapience.com)>

## 5.5 AI Tooling

Assistance from:

- Anthropic's [Claude](#) played a significant role updating and checking all C-code.
- Google's [Gemini](#) was especially helpful with Cython code.

---

<sup>1</sup> <https://abseil.io/about/philosophy#upgrade-support>

## PYTHON MODULE INDEX

### p

- `py_cidr.cidr_class`, [13](#)
- `py_cidr.cidr_file_class`, [26](#)
- `py_cidr.cidr_map`, [27](#)
- `py_cidr.pycidr_class`, [8](#)



## INDEX

### A

`add_cidr()` (*py\_cidr.cidr\_map.CidrMap* method), 28  
`add_cidrs()` (*py\_cidr.cidr\_map.CidrMap* method), 28  
`add_prefix_val()` (*py\_cidr.cidr\_map.CidrMap* method), 28  
`add_prefix_vals()` (*py\_cidr.cidr\_map.CidrMap* method), 28  
`address_ip_type()` (*py\_cidr.cidr\_class.Cidr* static method), 14  
`address_to_net()` (*py\_cidr.cidr\_class.Cidr* static method), 14  
`addresses_to_ips()` (*py\_cidr.cidr\_class.Cidr* static method), 14

### C

*Cidr* (class in *py\_cidr.cidr\_class*), 13  
`cidr_exclude()` (*py\_cidr.cidr\_class.Cidr* static method), 14  
`cidr_ip_type()` (*py\_cidr.cidr\_class.Cidr* static method), 15  
`cidr_is_subnet()` (*py\_cidr.cidr\_class.Cidr* static method), 15  
`cidr_list_compact()` (*py\_cidr.cidr\_class.Cidr* static method), 15  
`cidr_parts()` (*py\_cidr.pycidr\_class.PyCidr* static method), 9  
`cidr_range_split()` (*py\_cidr.cidr\_class.Cidr* static method), 15  
`cidr_set_prefix()` (*py\_cidr.cidr\_class.Cidr* static method), 16  
`cidr_to_net()` (*py\_cidr.cidr\_class.Cidr* static method), 16  
`cidr_to_range()` (*py\_cidr.cidr\_class.Cidr* static method), 16  
`cidr_to_range()` (*py\_cidr.pycidr\_class.PyCidr* static method), 9  
`cidr_type_network()` (*py\_cidr.cidr\_class.Cidr* static method), 17  
*CidrFile* (class in *py\_cidr.cidr\_file\_class*), 26  
*CidrMap* (class in *py\_cidr.cidr\_map*), 27  
`cidrs2_minus_cidrs1()` (*py\_cidr.cidr\_class.Cidr* static method), 17

`cidrs_exclude()` (*py\_cidr.cidr\_class.Cidr* static method), 17  
`cidrs_intersection()` (*py\_cidr.pycidr\_class.PyCidr* static method), 9  
`cidrs_split_type()` (*py\_cidr.cidr\_class.Cidr* static method), 17  
`cidrs_to_nets()` (*py\_cidr.cidr\_class.Cidr* static method), 18  
`clean_cidr()` (*py\_cidr.cidr\_class.Cidr* static method), 18  
`clean_cidr()` (*py\_cidr.pycidr\_class.PyCidr* static method), 9  
`clean_cidrs()` (*py\_cidr.cidr\_class.Cidr* static method), 18  
`clean_cidrs()` (*py\_cidr.pycidr\_class.PyCidr* static method), 10  
`compact()` (*py\_cidr.cidr\_class.Cidr* static method), 18  
`compact()` (*py\_cidr.pycidr\_class.PyCidr* static method), 10  
`compact_cidrs()` (*py\_cidr.cidr\_class.Cidr* static method), 19  
`compact_cidrs_to_cidrs()` (*py\_cidr.cidr\_class.Cidr* static method), 19  
`compact_nets()` (*py\_cidr.cidr\_class.Cidr* static method), 19  
`copy_cidr_file()` (*py\_cidr.cidr\_file\_class.CidrFile* static method), 26  
`create_private_cache()` (*py\_cidr.cidr\_map.CidrMap* static method), 28  
`ct_version()` (*py\_cidr.pycidr\_class.PyCidr* static method), 10

### E

`exclude_cidrs()` (*py\_cidr.pycidr\_class.PyCidr* static method), 10

### F

`fix_cidr_host_bits()` (*py\_cidr.cidr\_class.Cidr* static method), 19  
`fix_cidrs_host_bits()` (*py\_cidr.cidr\_class.Cidr* static method), 20

`fix_host_bits()` (*py\_cidr.pycidr\_class.PyCidr static method*), 10  
`format_host_bits()` (*py\_cidr.pycidr\_class.PyCidr static method*), 10

## G

`get_host_bits()` (*py\_cidr.cidr\_class.Cidr static method*), 20  
`get_host_bits()` (*py\_cidr.pycidr\_class.PyCidr static method*), 11

## I

`ip_to_address()` (*py\_cidr.cidr\_class.Cidr static method*), 20  
`ip_type()` (*py\_cidr.pycidr\_class.PyCidr static method*), 11  
`ip_version()` (*py\_cidr.cidr\_class.Cidr static method*), 21  
`ip_version()` (*py\_cidr.pycidr\_class.PyCidr static method*), 11  
`ipaddr_cidr_from_string()` (*py\_cidr.cidr\_class.Cidr static method*), 21  
`ips_to_addresses()` (*py\_cidr.cidr\_class.Cidr static method*), 21  
`is_rfc_1918()` (*py\_cidr.cidr\_class.Cidr static method*), 21  
`is_rfc_1918()` (*py\_cidr.pycidr\_class.PyCidr static method*), 11  
`is_subnet()` (*py\_cidr.pycidr\_class.PyCidr static method*), 11  
`is_valid_cidr()` (*py\_cidr.cidr\_class.Cidr static method*), 22  
`is_valid_cidr()` (*py\_cidr.pycidr\_class.PyCidr static method*), 11  
`is_valid_ip4()` (*py\_cidr.cidr\_class.Cidr static method*), 22  
`is_valid_ip6()` (*py\_cidr.cidr\_class.Cidr static method*), 22  
`is_valid_ipv4()` (*py\_cidr.pycidr\_class.PyCidr static method*), 11  
`is_valid_ipv6()` (*py\_cidr.pycidr\_class.PyCidr static method*), 12  
`items()` (*py\_cidr.cidr\_map.CidrMap method*), 28

## L

`lookup()` (*py\_cidr.cidr\_map.CidrMap method*), 28  
`lookup_all()` (*py\_cidr.cidr\_map.CidrMap method*), 28  
`lookup_both()` (*py\_cidr.cidr\_map.CidrMap method*), 29  
`lookup_lmp()` (*py\_cidr.cidr\_map.CidrMap method*), 29

## M

`merge()` (*py\_cidr.cidr\_map.CidrMap method*), 29  
module

`py_cidr.cidr_class`, 13  
`py_cidr.cidr_file_class`, 26  
`py_cidr.cidr_map`, 27  
`py_cidr.pycidr_class`, 8

## N

`net_exclude()` (*py\_cidr.cidr\_class.Cidr static method*), 22  
`net_is_subnet()` (*py\_cidr.cidr\_class.Cidr static method*), 22  
`net_range_split()` (*py\_cidr.cidr\_class.Cidr static method*), 23  
`net_to_cidr()` (*py\_cidr.cidr\_class.Cidr static method*), 23  
`net_to_range()` (*py\_cidr.cidr\_class.Cidr static method*), 23  
`nets_exclude()` (*py\_cidr.cidr\_class.Cidr static method*), 24  
`nets_to_cidrs()` (*py\_cidr.cidr\_class.Cidr static method*), 24  
`num_ips()` (*py\_cidr.pycidr\_class.PyCidr static method*), 12

## P

`print()` (*py\_cidr.cidr\_map.CidrMap method*), 29  
`py_cidr.cidr_class` module, 13  
`py_cidr.cidr_file_class` module, 26  
`py_cidr.cidr_map` module, 27  
`py_cidr.pycidr_class` module, 8  
`PyCidr` (class in *py\_cidr.pycidr\_class*), 9

## R

`range_to_cidrs()` (*py\_cidr.cidr\_class.Cidr static method*), 24  
`range_to_cidrs()` (*py\_cidr.pycidr\_class.PyCidr static method*), 12  
`read_cidr_file()` (*py\_cidr.cidr\_file\_class.CidrFile static method*), 26  
`read_cidr_files()` (*py\_cidr.cidr\_file\_class.CidrFile static method*), 26  
`read_cidrs()` (*py\_cidr.cidr\_file\_class.CidrFile static method*), 27  
`remove_rfc_1918()` (*py\_cidr.cidr\_class.Cidr static method*), 24  
`remove_rfc_1918()` (*py\_cidr.pycidr\_class.PyCidr static method*), 12  
`rfc_1918_cidrs()` (*py\_cidr.cidr\_class.Cidr static method*), 25  
`rfc_1918_cidrs()` (*py\_cidr.pycidr\_class.PyCidr static method*), 12

`rfc_1918_nets()` (*py\_cidr.cidr\_class.Cidr static method*), [25](#)

## S

`save_cache()` (*py\_cidr.cidr\_map.CidrMap method*), [29](#)

`set_prefix()` (*py\_cidr.pycidr\_class.PyCidr static method*), [12](#)

`sort()` (*py\_cidr.pycidr\_class.PyCidr static method*), [13](#)

`sort_cidrs()` (*py\_cidr.cidr\_class.Cidr static method*), [25](#)

`sort_ips()` (*py\_cidr.cidr\_class.Cidr static method*), [25](#)

`sort_nets()` (*py\_cidr.cidr\_class.Cidr static method*), [26](#)

`split_by_family()` (*py\_cidr.pycidr\_class.PyCidr static method*), [13](#)

`split_by_iptype()` (*py\_cidr.pycidr\_class.PyCidr static method*), [13](#)

`subnets_split()` (*py\_cidr.pycidr\_class.PyCidr static method*), [13](#)

## V

`version()` (*py\_cidr.cidr\_class.Cidr static method*), [26](#)

`version()` (*py\_cidr.pycidr\_class.PyCidr static method*), [13](#)

## W

`write_cidr_file()` (*py\_cidr.cidr\_file\_class.CidrFile static method*), [27](#)